Programming in Engineering 2024-202400656

3D Celestial Mechanics Simulation

Oliwier Piotr Sobczyk - s2597047

Supervisor: Igor Ostanin
i.ostanin@utwente.nl.

April, 2025

Faculty of Mechanical Engineering

UNIVERSITY OF TWENTE.

1 Introduction

This document presents the final assignment report for the Programming in Engineering course, focusing on the Python module. The assigned task was sample project 2, aimed at developing a prototype celestial mechanics code for a solar system with 100 planets orbiting a central Sun. The model incorporated gravitational influences on the central body, necessitating calculations of the system's center of gravity to monitor changes. Given the significant mass of the Sun compared to the planets, their individual impacts were minimal. Thus, the project included the three-body problem to highlight the interactions and perturbations among celestial bodies.

2 Problem Statement

This project focuses on simulating and visualizing gravitational interactions among celestial bodies in 3D space. The key challenge lies in precisely calculating the Newtonian gravitational motion[5] of multiple bodies over time while ensuring efficient performance and clear visualization. The system needs to take into account for different masses, positions, and velocities, update forces and positions with the Velocity Verlet method, and monitor the center of mass. FuncAnimation[2] is essential for animating each simulation step by recurrently invoking update() to refresh positions and trails, with init_func resetting the plot before starting.

2.1 Features

- Simulates interactions using Newtonian gravity.
- Visualizes 3D orbits of celestial bodies.
- Tracks and plots the trajectory of the system's center of mass (COM).
- Modular object-oriented code structure.
- Adjustable number of celestial bodies.

3 How To Run

Requirements: Python 3.x, NumPy, Matplotlib. Use "pip install numpy matplotlib" to install numpy and matplotlib if needed. Once ready run the main.py function or by "python main.py" in Terminal. To change the solution from 100 planets to a three body problem, open the config.py file and change the boolean **three_body_problem** to **True**.

4 Structure

This section explains the project's structure and gives an insight into each function's tasks.

- main.py is the main file from which the simulation can be run. It generates the planets, then creates "celestial" objects in celestialBody class found in body.py and then creates object in System class with created celestials, which contains the positions, masses of each planet and center of mass. Once the system is generated, it is forwarded with all celestials to the run_animation function stored in animation.py.

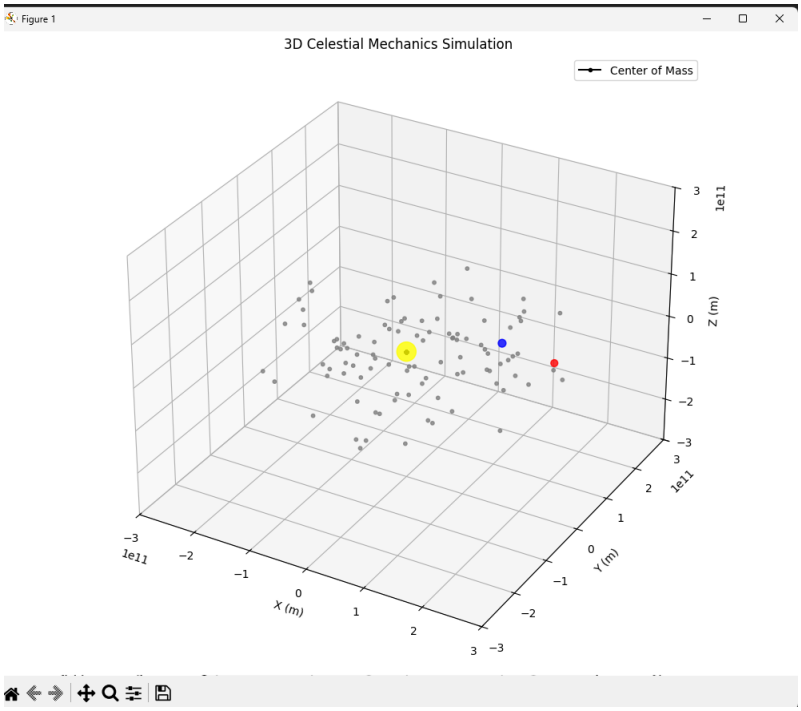

FIGURE 1: The result for the general case of the simulation

- config.py stores the constants like and variables, which can be changed by the user.
- system.py contains the system class, which generates the solar system with the given celestials objects.
- body.py this file contains the celestialBody class, which creates an object for a specified planet variables
- animation.py stores run_animation function accepts celestials list and system class and runs the animation while compiling the positions and center of mass

The core operational functions are **main.py** and **confing.py**, which allow the entire system to be managed and modified. Executing the main.py function simulates a general scenario with 100 planets orbiting the sun. After initialization, a figure similar to Fig.1 should appear.

4.1 config.py

This file contains constants: gravitational constant, plot limits, timestep (time between each update in seconds), mass of the sun, earth, mars and **variables**: skip (skips every n-th frame in animation, mass_inc (factor increasing the mass of earth and mars), three_body_problem which is a bolean giving the option to simulate 3 planets instead of 100. This file was made to avoid any changes in the main file. The user is free to change the variables.

$$MASS_EARTH = 5.972e24 * 10^{mass_inc}[kg] \tag{1}$$

4.2 system.py

Thus file contains the System class. It was made to make give the code more structure and have easier access to the positions of each planet and the position of center of Mass. This class accepts the list of all celestial objects. When initialised it contains all the celestials in a list, positions and masses of the celestials and the total mass of the system. It contains two functions: step and updateCoM. Step function calculates the new position, force and velocity of each iteration rising velocity Verlet algorithm[6].

$$
\begin{aligned}
\vec{v}_i\left(t+\frac{\Delta t}{2}\right) &= \vec{v}_i(t)+\frac{1}{2}\vec{a}_i(t)\Delta t \\
\vec{r}_i(t+\Delta t) &= \vec{r}_i(t)+\vec{v}_i\left(t+\frac{\Delta t}{2}\right)\Delta t \\
\vec{a}_i(t+\Delta t) &= \frac{\vec{F}_i(t+\Delta t)}{m_i} \\
\vec{v}_i(t+\Delta t) &= \vec{v}_i\left(t+\frac{\Delta t}{2}\right)+\frac{1}{2}\vec{a}_i(t+\Delta t)\Delta t
\end{aligned}
\tag{2}
$$

Full velocity Verlet integration step for position and velocity over time step Δt.

Before each step, the center of mass (CoM)[4] is calculated in the updateCoM function and each position is appended to an array for each x,y,z coordinate.

$$
\vec{r}_{\text{COM}} = \frac{\sum_i m_i \vec{r}_i}{\sum_i m_i} \tag{3}
$$

The center of mass is the weighted average of all body positions by mass.

4.3 body.py

To create the object for each planet celestialBody class was made in body.py. It is apart from the system class as the system analyses positions of the planets, but the interaction between them and planet's kinematics are obtained for each planet object.

4.3.1 init

Each planet class accepts its ID, radius[m],mass[kg], density[kg/m3], position[x,y,z] and velocity [vx,vy,vz] and initialises force and acceleration vector in 3D space.

4.3.2 updateForce(self,celestials)

This function accepts the self, and all other celestial objects in a list. Each time new force vector is initialised and the force is obtained from the interaction between each celestial using Eq.4.

$$
\vec{F}_i = \sum_j G \cdot \frac{m_i m_j}{\|\vec{r}_j - \vec{r}_i\|^3} \cdot (\vec{r}_j - \vec{r}_i) \tag{4}
$$

This equation calculates the gravitational force $\vec{F}_{ij}$ exerted on body i by body j, where G is the gravitational constant, m_i and m_j are their masses, and $\vec{r}_i$, $\vec{r}_j$ are their position vectors. Once the current force vector is known the acceleration is obtained with Eq.5

$$\vec{a}_i = \frac{\vec{F}_i}{m_i} \tag{5}$$

Acceleration is determined by the net force acting on a body and its mass.

4.3.3 updateVelocity(self,dt)

This function updates the velocity by adding the velocity change from acceleration.

4.3.4 updatePosition(self,dt)

This function updated the position after each step.

$$\vec{r}_i(t + \Delta t) = \vec{r}_i(t) + \vec{v}_i(t) \cdot \Delta t + \frac{1}{2}\vec{a}_i(t) \cdot \Delta t^2 \tag{6}$$

This equation updates the position vector $\vec{r}_i$ of body i at the next time step $t + \Delta t$ using its current position, velocity, and acceleration.

4.4 animation.py

The run_animation() function employs matplotlib's FuncAnimation[3] to create a 3D animation of celestial bodies moving over time. It prepares the plot by labeling axes and setting limits, initializes the celestial bodies' positions and trails, as well as the center of mass, and updates them frame-by-frame based on the physics simulation. Initially, the init()[1] function clears the visuals, while the update() function progresses the simulation over multiple time steps (skip), updating positions of the bodies, trails, and labels. FuncAnimation repeatedly calls update(), enabling a seamless depiction of the system's evolution.

References

[1] Animation in matplotlib. Accessed April 11, 2025.

[2] Introduction to matplotlib: 3d plotting and animations. Accessed April 11, 2025.

[3] Matplotlib Developers. matplotlib.animation.FuncAnimation. Accessed April 11, 2025.

[4] Wikipedia contributors. Center of mass — Wikipedia, the free encyclopedia, 2025. [Online; accessed 11-April-2025].

[5] Wikipedia contributors. Newton's law of universal gravitation — Wikipedia, the free encyclopedia, 2025. [Online; accessed 11-April-2025].

[6] Wikipedia contributors. Verlet integration — Wikipedia, the free encyclopedia, 2025. [Online; accessed 11-April-2025].